

PASTA Threat Model

Threat modeling a new sneaker marketplace app before launch

Project Description

For this activity, I worked through the seven stages of the PASTA (Process for Attack Simulation and Threat Analysis) framework to evaluate whether a new sneaker buying/selling app was ready to launch securely. I looked at the business goals behind the app, the technologies it relies on, how it handles data, the threats and vulnerabilities it could face, and the security controls that would reduce those risks.

Stage I: Define Business and Security Objectives

- The app needs to securely process real transactions between buyers and sellers, including several payment options, so payment handling has to be accurate and reliable.
- Data privacy is a major concern for the business — users need to trust that their personal and account information is being handled responsibly.
- Proper payment handling is tied to legal compliance, so the app needs to avoid mishandling financial data in a way that could create legal issues for the company.

Stage II: Define the Technical Scope

I would prioritize evaluating PKI first, since it directly protects the most sensitive data flowing through the app — payment details and personal information. The business specifically called out data privacy and payment handling as top concerns, so any weakness in how the AES and RSA encryption is implemented could expose users to fraud or the company to legal risk.

Stage III: Decompose the Application

Using the sample data flow diagram, I looked at the product search process as an example of how the app handles data. A user submits a search for sneakers for sale, which is sent to the product search process. That process queries the database and returns listings of current inventory back to the user. This same pattern — user input flowing into a process that reads from or writes to the database — repeats across the app's other features, like messaging and checkout, which is why the technologies protecting that data (PKI, SHA-256, and how SQL queries are written) matter so much.

Data flow diagram

Note: This data flow diagram represents a single process. Data flow diagrams for an application like this are normally much more complex.



Reference: sample data flow diagram — product search process

Stage IV: Threat Analysis

- External threat: A hacker could target the product search process with a SQL injection attack, attempting to manipulate the database query to access or leak sneaker listings and seller data.
- Internal threat: An employee could fall for a social engineering attempt, such as a phishing email, that tricks them into giving up login credentials and gives an attacker a way into the app's authentication system.

Stage V: Vulnerability Analysis

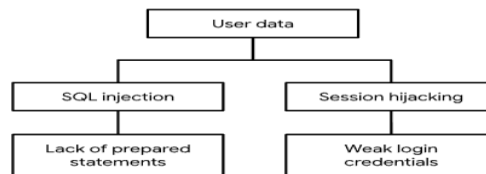
- The product search process may lack prepared statements in its SQL queries, which would leave it open to SQL injection attacks against the database.
- Weak login credential requirements could make it easier for an attacker to guess or brute-force a user's password and hijack their session.

Stage VI: Attack Modeling

The sample attack tree shows two ways an attacker could reach user data: through SQL injection, made possible by a lack of prepared statements, and through session hijacking, made possible by weak login credentials. Both attack paths trace back to the same root goal — gaining access to user data — which is why fixing the underlying vulnerabilities (unprepared SQL statements and weak credential requirements) is critical to closing off both branches.

Sample attack tree

Note: Applications like this normally have large, complex attack trees with many branches.



1

> ⋮

Google Slide

Reference: sample attack tree — paths to user data

Stage VII: Risk Analysis and Impact

- Use parameterized queries (prepared statements) instead of directly embedded SQL, to prevent SQL injection attacks on the database.
- Require multi-factor authentication (MFA) on user logins, so a stolen or guessed password alone isn't enough to hijack a session.
- Enforce strong password requirements and lock accounts after repeated failed login attempts, to make brute-force attacks much harder.
- Continue using AES/RSA encryption for sensitive data like payment details, and SHA-256 hashing for passwords, both in transit and at rest.

Summary

Working through PASTA's seven stages showed that the app's biggest risks trace back to two things: how it handles SQL queries and how it verifies user logins. Both were reachable through realistic attack paths in the attack tree, but both can be significantly reduced with well-known controls — prepared statements, MFA, strong password policies, and the encryption already planned for the app. With these controls in place, the app would be in a much stronger position to launch securely.